Efficient Algorithm Design of Optimizing SpMV on GPU

Genshen Chu
genshenchu@gmail.com
University of Science and Technology Beijing
Beijing, China

Yuanjie He
heyuanjie2021@163.com
University of Science and Technology Beijing
Beijing, China

Lingyu Dong
18811353956@163.com
University of Science and Technology Beijing
Beijing, China

Zhezhao Ding
dingzhezhao@126.com
University of Science and Technology Beijing
Beijing, China

Dandan Chen
chendandanustb@gmail.com
University of Science and Technology Beijing
Beijing, China

He Bai
baihe@xs.ustb.edu.cn
University of Science and Technology Beijing
Beijing, China

Xuesong Wang
13522551554@163.com
China Institute of Atomic Energy
Beijing, China

Changjun Hu
hu_ercis@163.com
University of Science and Technology Beijing
Beijing, China

ABSTRACT

Sparse matrix-vector multiplication (SpMV) is a fundamental building block for various numerical computing applications. However, most existing GPU-SpMV approaches may suffer from either long preprocessing overhead, load imbalance, format conversion, bad memory access patterns. In this paper, we proposed two new SpMV algorithms: *flat* and *line-enhance*, as well as their implementations, for GPU systems to overcome the above shortcomings. Our algorithms work directly on the CSR sparse matrix format. To achieve high performance: 1) for load balance, the *flat* algorithm uses non-zero splitting and *line-enhance* uses a mix of row and non-zero splitting; 2) memory access patterns are designed for both algorithms for data loading, storing and reduction steps; and 3) an adaptive approach is proposed to select appropriate algorithm and parameters based on matrix characteristics.

We evaluate our methods using the *SuiteSparse Matrix Collection* on AMD and NVIDIA GPU platforms. Average performance improvements of 424%, 741%, 49%, 46%, 72% are achieved when comparing our adaptive approach with CSR-Vector, CSR-Adaptive, HOLA, cuSparse and merge-based SpMV, respectively. In bandwidth tests, our approach can also achieve a high memory bandwidth, which is very close to the peak memory bandwidth.

CCS CONCEPTS

• **Theory of computation → Parallel algorithms**.

KEYWORDS

SpMV, sparse matrix, CSR, GPU, linear algebra

ACM Reference Format:
Genshen Chu, Yuanjie He, Lingyu Dong, Zhezhao Ding, Dandan Chen, He Bai, Xuesong Wang, and Changjun Hu. 2023. Efficient Algorithm Design of Optimizing SpMV on GPU. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23), June 16–23, 2023, Orlando, FL, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3588195.3593002

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HPDC '23, June 16–23, 2023, Orlando, FL, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0155-9/23/06…$15.00
https://doi.org/10.1145/3588195.3593002

1 INTRODUCTION

Sparse matrix-vector multiplication (SpMV) is a key linear algebra algorithm and has proven to be of great importance in computational science (such as Finite Element Method [2, 19, 37], Computational Fluid Dynamics [33, 41]), graph analytics[24, 39], deep learning [3] and many other fields. It is also a key component of many other linear algebra operations [40]. Since SpMV is on the critical path of many applications and math libraries, applications or solvers consuming large amounts of time on SpMV operations can directly benefit from performance improvement of SpMV optimization. Thus, many efforts have appeared for accelerating SpMV on different hardware, including CPU[30, 40], FPGA[14], GPU[27, 31, 32, 34, 38], SW26010 of Sunway Taihulight[7] and more. As a significant accelerator for general purpose computing, GPU provides great computation power and high memory bandwidth in data-parallelism situations. As a result, optimizing SpMV on GPU is essentially necessary.

Conceptually, the SpMV can be represented as $\boldsymbol{y} = \alpha \boldsymbol{A}\boldsymbol{x} + \beta \boldsymbol{y}$, where A is a sparse matrix, $\boldsymbol{x}$ and $\boldsymbol{y}$ are dense vectors, and α and β are scalars. For sparse matrix A, previous researches have developed more than fifty new storage formats, such as the well-known compressed sparse row (CSR), coordinate list (COO), ELLPACK (ELL), hybrid (HYB), and diagonal (DIA) formats[1, 13, 26]. Among them, the CSR format is predominantly used due to its efficient compression and good amenability [15, 29].

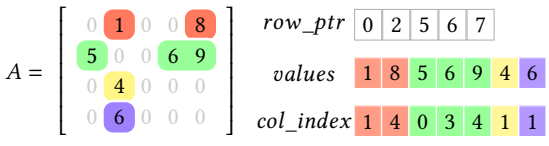


Figure 1: A sparse matrix and its CSR format

As shown in Figure 1, a sparse matrix A of m rows and n columns can be represented by three arrays: matrix entries array *values* of size NNZ (number of non-zeros), an integer array *col_index* of size

Algorithm 1: Naive Sequential SpMV $\boldsymbol{y} = \alpha \boldsymbol{A}\boldsymbol{x} + \beta \boldsymbol{y}$

```
1 for i ← 0 to m do
2     sum ← 0
3     for k ← row_ptr[i] to row_ptr[i + 1] do
4         sum ← sum + values[k] · x[col_index[k]]
5     y[i] ← β · y[i] + α · sum
```

NNZ positioning column coordinates of each entity, and another integer array *row_ptr* of size $m + 1$ which stores the beginning indexes of each row in array *values*. Then, a sequential CSR-based SpMV algorithm can be straightforward, as provided in Algorithm 1. Nevertheless, it can still be a great challenge to achieve high performance SpMV on GPU due to its bottlenecks of memory accesses [17, 21, 35] and data dependency.

SpMV is largely a memory bandwidth-bound operation, which needs to load $O(NNZ)$ non-zeros for only $O(NNZ)$ multiplication operations. Especially in CSR-based SpMV, one multiplication operation relies on memory accesses from three arrays (*values*, *col_index* and vector x). Moreover, CSR-based SpMV also suffers from the following challenges: 1) little data reuse: data on device memory are loaded into CU (Compute Unit) of the GPU only once, which leads to little data reuse (except some possible data-reuse in $\boldsymbol{x}$). This characteristic makes it challenging to utilize L1/L2 cache for data caching. 2) data dependency: to access vector $\boldsymbol{x}$, we must first load the value in array *col_index*; when summing the multiplication results of a row, the beginning and ending of a row rely on memory accesses of row offset array *row_ptr*. 3) reduction challenge: multiplication results of a row must be reduced and stored back to memory, which makes it difficult for the algorithm to process a row on multiple computational units. 4) workload balance issue on irregular matrix.

Numerous researches have been conducted to improve the performance of CSR-based SpMV (also called CsrMV) on GPUs. Their works can be categorized into two parts: first, one can adapt a new matrix storage format to use the GPU more efficiently (e.g., [22, 28, 32, 38]); second, one can optimize the CsrMV kernel by utilizing the characteristics of the GPU hardware. While the first approach presents concerns of matrix format compatibility and format conversion overhead [8]. For the second approach, a number of algorithms have been proposed, such as CSR-Scalar, CSR-Vector, CSR-Adaptive [8, 18], HOLA[36] and merge-based [30]. Nevertheless, existing algorithms may be limited in less consideration of memory access patterns, lack of matrix characteristics analysis, high overhead of preprocessing, or load imbalance. A more efficient SpMV solution is thus needed.

In this paper, we developed a new CsrMV library, SpMV-acc, for optimizing SpMV on both AMD GPU and NVIDIA GPU. In SpMV-acc, for different matrix characteristics, we proposed two new innovative CsrMV algorithms featuring removing or reducing preprocessing overhead, newly designed task dividing mode and memory access patterns. An adaptive strategy selection is then developed for selecting the appropriate SpMV algorithm and parameters among the above algorithms. Finally, we compared our solution with contemporary CsrMV parallelization strategies and the manufacturer's CsrMV under the SuiteSparse Matrix Collection[10]. This allows us to achieve 424%, 741%, 49%, 46% and 72% higher performance than CSR-Vector, CSR-Adaptive, HOLA, cuSparse and merge-based SpMV on average, respectively.

The key contributions of this paper are summarized as follows: 1) two novel SpMV algorithms are designed for GPU architecture: "line-enhance" and "flat"; 2) an adaptive strategy selection method is developed to select the appropriate SpMV algorithm and parameters among the above algorithms; and 3) the work is implemented and evaluated on two mainstream devices: AMD GPU and NVIDIA GPU, and can achieve impressive performance.

In this paper, we will present the previous state-of-the-art works in Section 2. In Section 3, the "flat" and "line-enhance" algorithm design as well as some optimizations and their adaptive approach will be introduced. The performance of our methods will be evaluated and discussed in Section 4. Finally, we present some conclusions and future work in Section 5.

2 STATE-OF-THE-ART CSR SPMV ON GPU

There exist many CSR-based SpMV algorithms for GPUs, but they can typically adhere to one of the following groups:

Row-splitting. As one of the earliest SpMV algorithms on GPUs, CSR-Scalar[5, 16] simply uses one thread for a row, which performs very poorly almost everywhere[18] due to its bad memory access pattern. An alternate approach, CSR-Vector [5], is to apply a wavefront (or called "warp" in CUDA) to calculate a matrix row. However, this approach can produce idle threads when it has short rows and can also cause load imbalance inside the workgroup (or called "thread block" in CUDA) when rows in a workgroup have large differences in the NNZ. Then, the improved CSR-Vector algorithm [1] [11] is developed and integrated into many libraries (e.g., cusp, rocSparse). It splits a wavefront into multiple groups of equal number of threads based on the average NNZ per row and makes a group of threads calculate a row jointly. However, this strategy still has discontinuous memory accessing issues among groups for both data loading and storing when processing short-row matrices, and it does not consider load balance among groups, especially for irregular matrices.

ACSR [4], a binning method, was proposed for grouping matrix rows into bins that have a similar number of non-zeros. Specifically, rows with NNZ that is larger than a predefined value are grouped into an individual bin $G1$. For each bin, a standalone kernel is launched to process the rows in the current bin. By grouping rows into bins, load balance can be considered. However, the binning method can suffer from the following disadvantages: 1) the overhead of generating bins from *row_ptr* cannot be neglected; 2) by splitting the rows into different kernels, the workload in each kernel can diminish, so that it may not make full use of the computation ability of the GPU in each kernel.

CSR-Stream [18] was proposed to assign a set of rows to a workgroup and use shared memory to achieve coalesced memory access in the data loading step. CSR-Adaptive [8] was later proposed and integrated CSR-Stream and CSR-Vector algorithms, in which CSR-Stream is used for short rows and CSR-Vector for long rows. Nevertheless, CSR-Stream and CSR-Adaptive both suffer from a

[1] In this paper, *CSR-Vector* always refers to the improved CSR-Vector because the original CSR-Vector in [5] can be considered a special case.

sequential preprocessing step on the CPU-side, whose execution time can be 10× longer than its SpMV kernel.

NNZ-splitting. Another approach, NNZsplitting, is proposed to assign equal NNZ to each workgroup. Steven Dalton et.al[9] (Dalton-SpMV) implemented the NNZ-splitting by decomposing the SpMV kernels into three phases: partition, reduction, and update. In the partition kernel, it uses binary search to locate the last row processed by each workgroup. In the reduction kernel, it loads the matrix and performs multiplication operations. Then the results are reduced using a workgroup-wide segmented scan, and the scan results are stored into the device memory except for the last discontinuous row that spans more than one workgroup. For the last discontinuous row in a workgroup, it stores the partial sum into an additional array r on device memory and then performs a device-wide segmented scan on r in the update kernel to update the final results. However, there are also drawbacks to this approach: 1) it increases the additional memory accessing overhead on r in the reduction and update kernel; 2) it requires a device-wide barrier before the update kernel; and 3) in the partition phases, the binary search is not quite efficient.

HOLA [36] is another implementation of the NNZ-splitting. HOLA uses a GPU-side preprocessing to generate records of which row each workgroup starts from and stores the records to device-side memory: array *blockRowStarts*. The SpMV kernel involves three shared memory arrays: *rowStarts* to store the row id of values assigned to each thread, a bit mask array *rowBits* to record the flag of the last entity in a row and an additional flag for processing empty rows. After the above shared memory arrays are correctly filled, matrix values and column indexes are loaded into registers in a coalesced way. Later, multiplication and summation are performed in a straightforward approach. Finally, the results are stored back using atomic operations because one row may be assigned to more than one workgroup.

In HOLA's implementation, shared memory is already used for arrays *rowStarts* and *rowBits*. This storage strategy leads HOLA to have to use registers for matrix values and column indexes storing when loading matrix data. However, this approach may potentially use more register resources. Thus, the number of active wavefronts running on execution units (EU) is limited. This attribute could cause a bad effect on hiding memory access latency by involving enough calculation and prevent HOLA from achieving higher performance.

Hybrid splitting. Merge-based SpMV [30] considers load balance not only on matrix *values*, *col_index* but also on the output vector $\boldsymbol{y}$ and row offset *row_ptr*. It indeed provides better workload balance. However, there are also downsides to the merge-based approach: first, as commented in [36], threads have to perform searches for their starting row/non-zero; second, for matrices with long rows, the weighting of loading and storing output vector is very small. Thus, in this case, involving output vector into workload balance may bring larger processing overhead when compared to the benefit of better load balance.

3 METHODS

The design principles of our SpMV-acc are: no format conversion overhead, without or with less preprocessing overhead, and high memory bandwidth usage. For the first two principles, we directly operate SpMV on the CSR format and avoid involving preprocessing. The high memory bandwidth is mainly contributed by the design of memory access patterns and workload balance in our algorithms.

In this section, our complete SpMV solution is described, which introduces two newly designed SpMV algorithms, *flat* and *line-enhance*, and an adaptive strategy to select a SpMV algorithm and parameters based on matrix characteristics.

3.1 Flat Method

The *flat* SpMV algorithm is an NNZ-splitting algorithm. Its basic idea is similar to HOLA and Dalton-SpMV, in which it assigns equal NNZ to each workgroup. One of the main differences from HOLA is that our *flat* algorithm does not involve bit-mask or *rowStarts* array in LDS (local data store or shared memory). Instead, the LDS is used as a buffer for utilizing the memory coalescing feature of GPUs. In this way, *flat* would consume fewer registers for data loading, which can efficiently hide latency of memory access via enough computation (in our experiments on ROCm, *flat* uses 16 vector general purposed registers (VGPRs), while VGPRs consumed by HOLA are 40).

The main challenge here is to reduce the results to correct rows while taking into account performance at the same time. We must find an approach to search the starting and ending index for each row under the NNZ-splitting strategy and should process the case when a row spans multiple workgroups. For starting and ending index searching, HOLA uses an additional array *rowStarts* and bit mask array *rowBits* to achieve this, but it could involve more overhead to access the shared memory. In *flat*, we developed a method to calculate the indexes directly based on the *break point* array. For the case of spanning multiple workgroups of a row, we perform memory stores via atomic operation.

The detailed *flat* algorithm can be broken down into five major steps: 1) GPU-side preprocessing for break point generation; 2) initialization of the SpMV kernel; 3) matrix data loading and multiplication; 4) parallel reduction of rows; and 5) store results back to device memory.

Algorithm 2: Preprocessing Algorithm of Flat Method

```
Input: WGS - workgroup number in the kernel
Input: THREADS - thread number in a workgroup
Input: STRIDE - break stride, NNZ processed by a workgroup
/* g_thread_id: global thread id in the GPU kernel */
if g_thread_id == 0 then bp[0] ← 0          ▷ bp: break point array
for i ← g_thread_id to m step WGS · THREADS do
    cur_row_bp_id ← row_ptr[i]/STRIDE
    next_row_bp_id ← row_ptr[i + 1]/STRIDE
    if cur_row_bp_id ≠ next_row_bp_id then
        for j ← cur_row_bp_id + 1 to next_row_bp_id + 1 do
            bp[j] ← i
        if row_ptr[i + 1] (mod STRIDE) == 0 then
            bp[next_row_bp_id] ← bp[next_row_bp_id] + 1
```

3.1.1 Break Point Generation. When the tasks are divided by NNZ, a row can span multiple workgroups. Moreover, when a workgroup

stores results back to array $\boldsymbol{y}$, it must know the exact row index. In other words, a workgroup must know which rows the data assigned belong to. A preprocessing step is thus needed to calculate the starting row of each workgroup before running the SpMV kernel.

The basic idea here is to iterate over the array row_ptr and calculate a row id with which each workgroup is associated. We use an additional device array bp (break point array) to store the association between workgroups and rows. The array bp is defined as follows: 1) its length equals the workgroup number in the core SpMV kernel, and 2) $bp[i]$ represents **the row id of the first non-zero processed by workgroup** i.

In *flat* implementation, this preprocessing step can be parallel on the GPU-side due to no data dependency between rows. Algorithm 2 provides a pseudocode for the GPU-side to achieve the above goal. It iterates over all rows in the matrix. In each iteration, we consider row i and row $i+1$ and calculate the workgroup id ($cur_row_bp_id$ and $next_row_bp_id$) that the first non-zeros from both rows fall in, respectively. If the two workgroup ids are not equal, which means the first non-zeros in row i and $i+1$ are assigned to different workgroups, then elements in array $bp[cur_row_bp_id+1]$ to $bp[next_row_bp_id]$ would all be filled with value i (line 6 to 7) because the first non-zero processed by workgroup $cur_row_bp_id+1$ to $next_row_bp_id$ all come from row i (except a special case for $bp[next_row_bp_id]$). In fact, if the first non-zero in row $i+1$ corresponds exactly to the first non-zero assigned to workgroup $next_row_bp_id$, the first non-zero processed by workgroup $next_row_bp_id$ comes from row $j+1$, instead of i. Thus an additional adjustment is needed to process the kind of special cases (line 8 to 9).

The preprocessing kernel is a lightweight kernel and can be executed on the GPU-side. In this kernel, the memory access of row_ptr is adjacent among threads. In addition, in our design, each workgroup can typically handle approximately thousands of non-zeros. Thus, the memory accessing amount of break point array bp can just be about $NNZ/STRIDE$ times, which is quite small in most cases when comparing memory accessing of row_ptr. In fact, the preprocessing we used is very similar to HOLA's preprocessing step because we both obey the basic idea of assigning an equal number of non-zeros to workgroups.

3.1.2 Initialization of SpMV Kernel. From this step, *flat* starts the SpMV calculation in the core SpMV kernel. The whole core SpMV kernel is presented in Algorithm 3 and Figure 2. Before starting, we denote the thread number in each workgroup as *THREADS*, which is usually an integer multiple of the wavefront size or warp size. We also mark the number of non-zeros processed by a workgroup as N and ensure $N = R \times THREADS$, where R is a small positive integer (e.g., 2 or 4). In the initialization step, it only needs to complete one thing: on-chip shared memory of size N is allocated for storing temporary results of multiplication later.

3.1.3 Data Loading and Multiplication. In this step, non-zeros and column indexes assigned to the current workgroup, as well as corresponding elements in vector $\boldsymbol{x}$, are directly loaded into registers from device memory (line 7-11). This is an advantage of our *flat* algorithm, which can load the whole matrix in a completely coalesced manner and without any data exchange or movement among threads or wavefronts. Moreover, loop unrolling optimization can

Algorithm 3: Core Flat algorithm with direct reduction

```
   Input: WGS - workgroup number in the kernel
   Input: THREADS - thread number in a workgroup
   Input: R - an integer related to the size of shared memory
   Input: bp[] - break point array on device memory
1  N ← THREAD · R // NNZ processed by a workgroup
2  last_row_idx ← row_ptr[m]
3  workgroup_row_idx_start ← N · workgroup_id
4  break_point_idx ← workgroup_id
5  for k ← workgroup_row_idx_start to last_row_idx do
6      syncthreads()
       /* load values and col_inx, and perform
          multiplication                                  */
7      for i ← 0 to R do
8          shared_idx ← thread_id_in_workgroup + i · THREADS
9          idx ← min(k + shared_idx, last_row_idx − 1)
10         temp ← values[idx] · x[col_idx[idx]]
11         shared_mem[shared_idx] ← temp
12     syncthreads()
       /* calculate reduction id                          */
13     reduce_row_start ← min(bp[break_point_idx], m)
14     reduce_row_end ← min(bp[break_point_idx + 1], m)
15     if reduce_row_end == 0 then  reduce_row_end ← m
16     if row_ptr[reduce_row_end] (mod N) == 0 or
         reduce_row_start == reduce_row_end then
17         reduce_row_end ← min(reduce_row_end + 1, m)
18     reduce_row_id ← reduce_row_start + thread_id_in_workgroup
19     bp_nnz_id ← break_point_idx · N      ▷ break point nnz id
       /* reduce and store results back                   */
20     while reduce_row_id < reduce_row_end do
21         sum ← 0
22         reduce_idx_start ←
             max(0, row_ptr[reduce_row_id] − bp_nnz_id)
23         reduce_idx_end ←
             min(N, row_ptr[reduce_row_id + 1] − bp_nnz_id)
24         for i ← reduce_idx_start to reduce_idx_end do
25             sum ← sum + shared_mem[i]
26         AtomicAdd(y + reduce_row_id, sum)
27         reduce_row_id ← reduce_row_id + THREADS
28     break_point_idx ← break_point_idx + WGS
```

be applied to the loop at line 7, which allows compilers to adjust the position of memory access instructions and other instructions to hide memory access latency. In our experiments, loop unrolling can lead to approximately 5% to 10% performance improvement.

After all the data are loaded, each thread in the workgroup executes a multiplication instruction and writes the multiplication results into the correct position in shared memory (see LDS in Figure 2). Here, correct position means that the multiplication result from the ith non-zero processed by the current workgroup will be written to the ith entity of the current workgroup's shared memory, as illustrated in Figure 2.

3.1.4 Reduction. The underlying problem left is to reduce the results to correct rows. A straightforward approach is making each

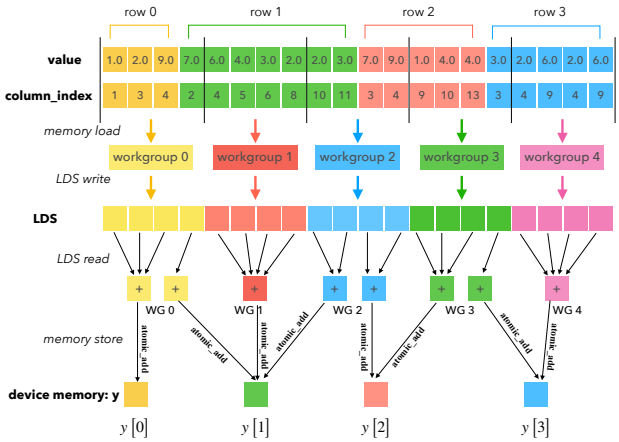


Figure 2: An example diagram of *Flat* strategy. Here, "WG" means "workgroup".

thread responsible for the reduction of one row. Reduction can be carried out in two main steps: 1) calculate the row range for reduction and 2) perform the summation operation.

The row range calculation procedure starts from the *bp* array, which can provide the row id of the first non-zero processed by the current and the next workgroup (see lines 13 and 14 in Algorithm 3). Then, a row range for the current workgroup, [$reduce_row_start$, $reduce_row_end$), can be given. In particular, some special cases should be handled when generating the reduction row range, including 1) the current workgroup is the last workgroup on the GPU (line 15); 2) the first non-zero in the next workgroup is the first non-zero of some row (line 16); and 3) the current workgroup only processes non-zeros from one row (line 16).

In the summation procedure (lines 20 to 25), each partial row is assigned to a thread in the current workgroup. For example, in Figure 2, workgroup 0 contains non-zeros from two rows, then only threads 0 and 1 in workgroup 0 are active for summation. Each thread reads the multiplication results of its row from LDS and adds up to the thread's private register. Note that the loop in line 20 is designed to process the case when there are more rows assigned to the workgroup than threads in the workgroup (for example, there are many empty rows, which also indicates that *flat* can implicitly handle empty rows. As a comparison, HOLA requires special code branches for empty rows, which may decrease performance[36]).

In practice, considering the overhead of LDS reading, we may not directly reduce one row using only one thread. Instead, we can group multiple threads as a vector to perform the parallel reduction of one row. The idea here is very similar to CSR-Vector SpMV. However, CSR-Vector is designed to load data from device memory.

3.1.5 Store Results Back to Device Memory. As the sum of each row processed by the current workgroup is located in each thread or vector, the thread/vector can directly store results to device memory. The atomic add operation for the float point number is applied to store the results back because a row may span multiple workgroups.

Flat, HOLA, and Dalton-SpMV are all based on NNZ-splitting. In the preprocessing step, Dalton-SpMV uses binary search to generate indexes of start rows for each workgroup, in which $\frac{NNZ}{NNZ_PER_WORKGROUP}$ threads are launched for searching. Each thread can access row_ptr more than one time (approximately $\log(m)$ times) and the memory access pattern can be scattered (two adjacent threads may fall into different searching ranges). While *flat* and HOLA use a more efficient manner to locate the starting row of each workgroup, in which m threads are involved to check whether a row is the starting row of some workgroup. In this approach, the memory access pattern can be coalesced and each thread can access the row_ptr array only once.

In the calculation and reduction step, HOLA must search the starting row for each thread in a workgroup and write the records into LDS (the arrays $rowStarts$ and $rowBits$ in its algorithm). In the *flat* method, the above searching step is omitted. Instead, we can directly calculate the row id for the reduction vector or thread via the vector or thread id as the offset (see line 18 in Algorithm 3). Both *flat* and Dalton-SpMV write the multiplication results to LDS and perform reduction from LDS space. For the Dalton-SpMV, the additional overhead can as follows: 1) it uses a workgroup-wide segmented scan operation to reduce values from LDS. While the *flat* method directly uses vector for final results reduction, which can be more efficient . 2) Dalton-SpMV needs to generate row indexes corresponding to each nonzero for the later workgroup-wide scan. For handling the row spanning multiple workgroups, the *flat* method, as well as HOLA, directly perform *AtomicAdd* on the result $\boldsymbol{y}$. While the Dalton-SpMV involves an additional update kernel that requires a device-wide barrier and needs to perform a device-wide segmented scan in the update kernel. The performance comparisons and discussions of *flat*, HOLA, and Dalton-SpMV are presented in Section 4.1.

Similar to Dalton-SpMV, the merge-based method also relies on binary searching in its preprocessing step to position the 2D-path coordinates for each workgroup. Compared with *flat*, the merge-based method may have additional overhead from at least one of the following parts: 1) binary search in the preprocessing step; 2) on-the-fly search of the merge-path; and 3) reduce-value-by-key primitives for reduction on workgroup-wide (read from LDS) and device-wide (load from global memory).

3.2 Line-enhance Method

NNZ-splitting distributes equal non-zeros to each workgroup. Nevertheless, it can also involve some unnecessary overhead, including preprocessing, *bp* array accessing, and atomic operation for storing data back. Thus, we designed another CsrMV algorithm based on a hybrid of row-splitting and NNZ-splitting, called *line-enhance*, to overcome the above shortcomings from *flat*. The basic idea of *line-enhance* is to assign the entire rows to workgroups and distribute equal non-zeros to threads in each workgroup.

As non-zeros are assigned to threads inside workgroup, a row may be cut into more than one part, which can be a challenge for row reduction and results storing back steps. We work on this problem by involving a novel reduction method that is inspired by the *flat* algorithm (see "reduction" section). For the results storing back step, the *line-enhance* algorithm does not involve expensive atomic operations because an entire row is assigned to a workgroup and there is no row spanning multiple workgroups. This strategy

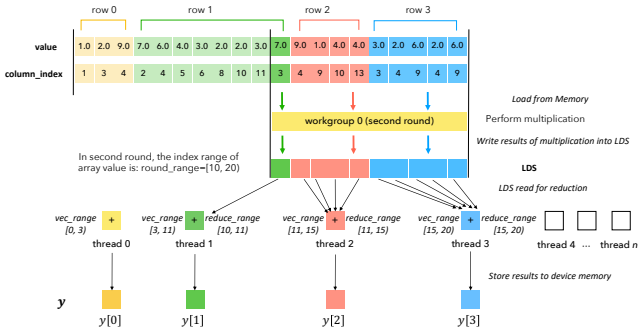


Figure 3: An example diagram of the *Line-Enhance* strategy. Here, N=4 and V=1.

can effectively avoid the above disadvantages from the *flat* strategy or other workgroup-level NNZ-splitting strategies.

The details of *line-enhance* can be described in five parts: 1) task partitioning and initialization; 2) round calculation and round loop; 3) multiplication; 4) reduction; and 5) results storing.

Algorithm 4: Core Line-enhance SpMV algorithm

Input: *THREADS* - thread number in a workgroup
Input: N - rows processed by a workgroup
Input: V - thread number in a vector (vector is used for reduction)
Input: R - an integer related to the size of shared memory

```
   /* tasks partitioning & initialization                    */
1  workgroup_row_begin ← workgroup_id · N
2  workgroup_row_end ← min(workgroup_row_begin + N, m)
3  workgroup_row_idx_start ← row_ptr[workgroup_row_begin]
4  workgroup_row_idx_end ← row_ptr[workgroup_row_end]
5  rounds ← ⌈ (workgroup_row_idx_end − workgroup_row_idx_start) / (R · THREADS) ⌉
6  shared_mem[R · THREADS], sum ← 0
   // rounds loop with NNZ splitting
7  for r ← 0 to rounds do
8      round_inx_start ← workgroup_row_idx_start +
         r · R · THREADS
9      round_inx_end ← min(round_inx_start + R ·
         THREADS, workgroup_row_idx_end)
10     syncthreads()
11     i ← round_inx_start + thread_id_in_workgroup
12     for k ← 0 to R do
13         if i < workgroup_row_idx_end then
14             tmp ← values[i] · x[col_ind[i]]
15             shared_mem[i - round_inx_start] ← tmp
16         i ← i + THREADS
17     syncthreads()
       /* reduce from shared memory                          */
18     sum ← sum + VecReduce(shared_mem)
   /* store results back                                     */
19 if V ≠ 1 then sum ← VecPrefixSum(sum, V)
20 reduce_row_id ← workgroup_row_begin + vec_id_in_workgroup
21 if reduce_row_id < workgroup_row_end and tid_in_vec == 0 then
22     y[reduce_row_id] ← α · sum + β · y[reduce_row_id]
```

3.2.1 Task Partitioning and Initialization. In task partitioning, we simply assign equal rows, denoted as N, to each workgroup. For workgroup i, the range of its processed rows can be initialized as $row_range = [workgroup_row_begin, workgroup_row_end)$ (the row $workgroup_row_end$ is not processed by workgroup i), as demonstrated in Algorithm 4. For instance, in Figure 3, each workgroup is assigned to process 4 rows. In this case, row_range for workgroup 0 can be $[0, 5)$ (the 5th row is processed by the next workgroup).

Similar to the *flat* method, in the initialization step, the on-chip shared memory of size $R \times THREADS$ is allocated for buffer usage, where R is a small and positive integer.

3.2.2 Rounds Calculation. In *line-enhance*, multiple rounds are applied to the multiplication and reduction steps. In each round, $R \times THREADS$ non-zeros from the matrix are loaded and processed. Then, the total rounds of the current workgroup can be easily determined via the total non-zeros assigned to this workgroup and NNZ processed in each round (see lines 3-5).

Here, data accessing from row_ptr can be memory coalesced because all threads in a workgroup access the same address. Another alternative approach is that the first thread in the wavefront loads row_ptr into its register and then broadcasts it to other threads in this wavefront by a shuffle instruction. Interestingly, it would not bring an obvious performance drop to us when compared with the former approach. This can be explained by the benefit of memory coalescing, even if all threads in the workgroup perform memory accesses (at the CUDA code level).

After the total round is calculated, a loop is created to iterate all rounds. In each round, an equal number of non-zeros is loaded and processed.

3.2.3 Multiplication. In each round, to load data of the matrix from device memory, a range of loading indexes for threads in the entire workgroup, $round_range$, is first determined, where $round_range = [round_inx_start, round_inx_end)$ (see lines 8 and 9). $round_inx_start$ and $round_inx_end$ can be determined by the base index of the current workgroup $row_ptr[workgroup_row_begin]$ plus an offset to the current round.

Using the round range, all threads in the current workgroup can jointly load at most $R \times THREADS$ entities from array $values$ and col_inx, as well as vector $\boldsymbol{x}$, into their registers. Note that loop unrolling optimization in *flat* can also be migrated to the *line-enhance* algorithm for latency hiding by adjusting the instructions' position. Obviously, addresses of this loading procedure are adjacent among threads in all wavefronts of each workgroup; thus, memory accesses can be coalesced. In addition, the workload is also balanced among threads at the workgroup level.

In each round, after memory loading of an entity set (including a value from $values$, col_inx and $\boldsymbol{x}$) is completed in the thread, multiplication can be executed, and the multiplication result can be buffered to the correct location in LDS (lines 14 and 15). After a barrier of waiting for the LDS writing operation to be completed, the algorithm then moves to the next stage.

3.2.4 Reduction. Similar to the CSR-Vector and *flat* algorithms, threads in the workgroup are divided into multiple vectors, and make each vector reduce one row. We denote the thread number in a

Algorithm 5: VecReduce: Reduction in Line-Enhance

```
/* vec_i: vector id in the current workgroup;
   tid_in_vec: thread id in the current vector.      */
1 local_sum ← 0
2 if workgroup_row_begin + vec_i < workgroup_row_end then
3     reduce_range ← vec_range ∩ round_range
4     if reduce_range ≠ ∅ then
5         R_begin ← max(I_begin, round_inx_start)
6         R_end ← min(I_end, round_inx_end)
7         for j ← reduce_start + tid_in_vec to reduce_end step V do
8             local_sum ← local_sum + shared_mem[j -
                round_inx_start]
9 return local_sum
```

vector as V, where V is a power of two and meets the constraints of $THREADS/V \leq N$. In particular, when V equals 1, only one thread is used for reducing a row. This configuration can be efficient when a matrix contains many short rows.

We consider a range, $vec_range = [I_{begin}, I_{end})$, for the ith vector in the workgroup, where $I_{begin} = row_ptr[r]$, $I_{end} = row_ptr[r+1]$ and $r = workgroup_row_begin + i$. We also consider another range $reduce_range = vec_range \cap round_range$. If the intersection $reduce_range$ is empty, it means that, in the current round, there is no entity needing reduction for vector i and threads on vector i should be inactive. Otherwise, set $reduce_range$ is nonempty (denoted as $reduce_range = [R_{begin}, R_{end})$), and a reduction is needed for the row r associated with vector i. As an example, the vec_range and $reduce_range$ of each vector, as well as $round_range$, are exhibited in Figure 3 ($V = 1$).

To perform reduction in vector i (vec_i in Algorithm 5), it just needs to sum all values in LDS from index R_{begin} to R_{end} and save the results to register $local_sum$. This step can be parallelized by V threads in the vector, as presented in Algorithm 5.

3.2.5 Results Storing. As the partial sum is located in each thread inside the vector, we must first move and add the sum to the first thread from all other threads at the vector level. Thus, a prefix sum ([6, 20]) is performed at the vector level to achieve this goal (line 19 in Algorithm 4). Then, the results can be directly stored back to device memory by the first thread in the active vector. It is worth noting that a memory optimization here can be carried out to overlap memory loading and calculation by moving the step of array y data loading before the prefix sum.

The *line-enhance* algorithm assigns the entire rows to workgroups and performs local load balance inside the workgroup. Even if the NNZ of rows inside a workgroup vary greatly, *line-enhance* can assign equal non-zeros to each thread inside the workgroup to ensure the load balance among threads. Furthermore, the *line-enhance* algorithm can also load the matrix in a coalesced manner because threads inside a workgroup load the continuous elements in the *values* and *col_index* array. While other row-splitting CSR-Vector and binning methods do not guarantee the above two characteristics. Especially when the matrix row is short (for example, NNZ per row < 8), the CSR-Vector method may perform poorly [18] because threads in two adjacent vectors may access discontinuous memory space.

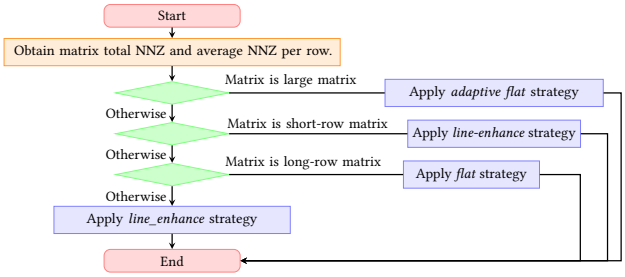


Figure 4: Flowchart of the adaptive strategy

3.3 Adaptive Selection of Strategies and Parameters

3.3.1 Strategies Selection. We have described two different SpMV algorithms, *flat* and *line-enhance*, which are advantageous for different cases. We can combine the two algorithms into an adaptive approach for more efficient SpMV calculation. Figure 4 describes our adaptive approach, in which the *flat* algorithm is selected when the matrix is large enough or it is a long-row matrix, and the *line-enhance* algorithm is applied for other cases. Here, the large matrix is defined as the matrix with NNZ > 1 000 000 ($1M$), the long-row matrix is defined as the matrix with NNZ per row > 128, and the short-row matrix is defined as the matrix with NNZ per row < 8. This is because *flat* provides a better load balance for long-row matrices, but it may suffer from a large percentage of preprocessing overhead for small matrices or a significant amount of atomic operations for short-row matrices. While *line-enhance* provides good performance for small matrices and short-row matrices due to its preprocessing-free design and assignment of entire rows.

3.3.2 Parameters Selection. After the SpMV algorithm is determined, the parameters in the algorithm should also be adaptively tuned based on matrix characteristics. For example, for the reduction step in both *flat* and *line-enhance*, the thread number in vector (parameter V) is determined via the average NNZ per row to achieve the best reduction performance. In addition, parameters R (used to control rounds number), $THREADS$ (thread number in a workgroup) and N (rows per workgroup) in *line-enhance*, and R (used for denoting NNZ processed by a workgroup) and $THREADS$(thread number in a workgroup) in *flat*, are all tuned via a combination of empirical and theoretical approaches.

For the *line-enhance* algorithm, the parameters R and $THREADS$ are fixed to 2 and 512, respectively. Then, the shared memory used by a workgroup is also fixed. In this case, the shared memory size is also very appropriate for GPU scheduling. If the average NNZ per row of a matrix is larger than a given value (for example, 24), we treat the matrix as a *more-nnz-per-row* matrix. Then, fewer rows (for example $N = 64$) can be processed by a workgroup (a row has more non-zeros), and it needs more threads in a vector (for example $V = 4$) for reducing a row. Otherwise, more rows (for example $N = 128$) and fewer threads in a vector (for example $V = 1$) can be

applied. For the *flat* algorithm, the idea of the adaptive parameters is similar to the *line-enhance* algorithm.

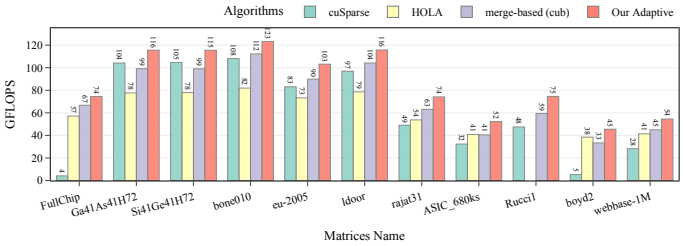
4 EVALUATION

To evaluate our approach, we benchmarked the *SuiteSparse Matrix Collection* (the University of Florida Sparse Matrix Collection) [10], which contains 2893 matrices (statistical data in November 2021) of different sizes in a variety of application fields. Eventually, 2638 matrices were chosen by NNZ from 10^3 (1k) to 10^9 (1G), covering 91% of the matrices in the entire SuiteSparse Matrix Collection.

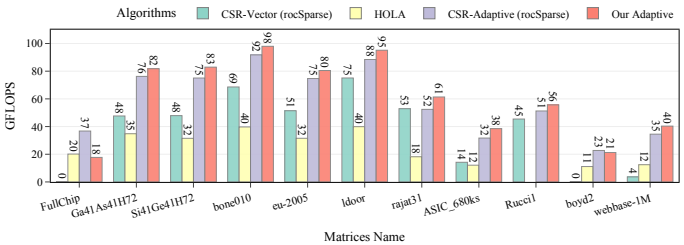
The performance tests were performed on both AMD GPU and NVIDIA GPU. Table 1 details their important characteristics. All performance results in this paper are from double-precision floating-point tests.

Table 1: specifications of AMD and NVIDIA GPU in our tests

Environment	AMD GPU (ROCm)	NVIDIA GPU (CUDA)
Architecture	GCN5-GFX906-VEGA	Tesla V100
CUs or SMs	64	80
Peak Memory Bandwidth	800GB/s	900GB/s
Memory Type	HBM2	HBM2
Memory Clock Rate	800 MHz	877 MHz [23]
LDS Size per CU/SM	64 KiB	up to 96 KiB
Compilers	hipcc 3.9.1 (clang 12 based)	CUDA 10.2 (gcc 7.4.0 based)



(a) Our adaptive versus other approaches on NVIDIA



(b) Our adaptive versus other approaches on ROCm. Note: CSR-Adaptive requires up to 100× additional preprocessing time on the CPU-side (the plot does not include the preprocessing time).

Figure 5: Speedup of our adaptive method versus other algorithms for commonly tested matrix cases. The number over the bar is the FLOPS of each case. All matrix cases have been widely adopted in previous researches (e.g., CSR-Adaptive[8, 18], CSR5 [28], yaSpMV [38]).

4.1 SpMV Performance

4.1.1 SpMV Performance of the Adaptive Approach. First, the performance of our adaptive approach was compared with HOLA [36], CSR-Vector (integrated in rocSparse), CSR-Adaptive[8] (integrated in rocSparse) on the AMD GPU platform, and HOLA, cuSparse and merge-based[30] (integrated in *cub* library, v1.12.1) on the NVIDIA Tesla V100 platform. rocSparse and cuSparse are the manufacturer's sparse computation libraries for the AMD ROCm platform and NVIDIA platform, respectively. cub library as well as merge-based SpMV in it is also integrated as a part of the CUDA toolkit.

Figure 6 and Figure 7 summarize the relative SpMV speedup our adaptive approach achieved over other competing approaches on both platforms. The main indicators of the relative SpMV speedup are listed in Table 2. The performance of commonly compared matrices is exhibited in Figure 5. Overall, our adaptive approach can outperform other evaluated methods (apart from cuSparsse) in **more than 90% of matrix cases** and can surpass cuSparse in **81.35% of small matrix cases** and **70.81% of large matrix cases**. Furthermore, for 97.26% of small matrix cases and 94.59% of large matrix cases, our adaptive approach can achieve the best performance among HOLA, CSR-Adaptive, and CSR-Vector on the ROCm platform. However, the corresponding best ratios are 73.61% and 66.38% on the NVIDIA platform among HOLA, cub, and cuSparse algorithm/library, but they are still competitive.

The classical CSR-Vector algorithm, which is widely used in many linear algebra libraries (e.g., clSPARSE, cusp, rocSparse), was compared with our adaptive approach on the ROCm platform. The results show that our method achieves up to 319.1× speedup versus CSR-Vector (rocSparse) on highly irregular matrices and leads approximately 424% performance improvement over CSR-Vector in rocSparse on average.

For merge-based SpMV, the default SpMV implementation in *cub* lib, our method provides average speedups of 2.75 and 1.72 for small and large matrix cases, respectively. Please note that both HOLA and merge-based SpMV do not consider the term $\beta \boldsymbol{y}$ in SpMV calculation (they only calculate $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$, instead of $\boldsymbol{y} = \alpha \boldsymbol{A}\boldsymbol{x} + \beta \boldsymbol{y}$). In other words, the real speedup of our method versus merge-based SpMV should be higher if they both ignore or both consider the term $\beta \boldsymbol{y}$.

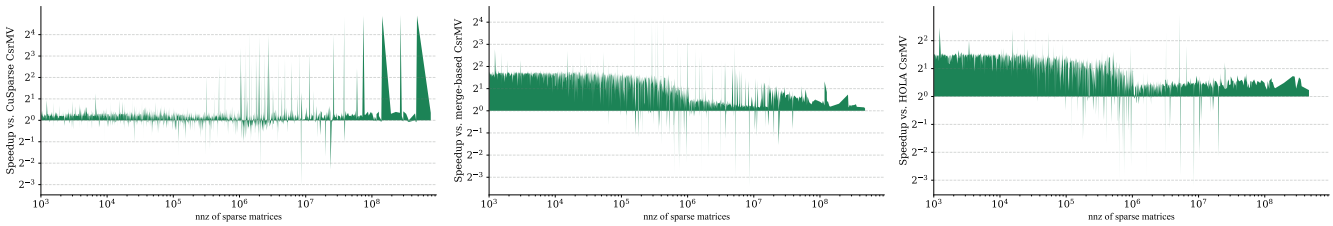
HOLA was tested on both the ROCm and NVIDIA platforms. On average, we can achieve more than 30% performance improvement on the ROCm platform and 129%(small matrix cases)/49%(large matrix cases) performance improvement on the NVIDIA platform when comparing our adaptive method with HOLA.

cuSparse achieves the closest performance to our adaptive approach on the NVIDIA platform. In small matrix cases, our approach provides a slight performance improvement (12% on average) against cuSparse but still consistently achieves a better performance in 81.35% of cases. For large matrix cases, whose performance is more significant in practice, the mean speedup increases to 1.46. Furthermore, our approach can achieve quite a high speedup (e.g., > 2×) for a significant number of large matrix cases, as shown in Figure 6(a).

Table 2: Relative speedup of our adaptive SpMV over competing approaches

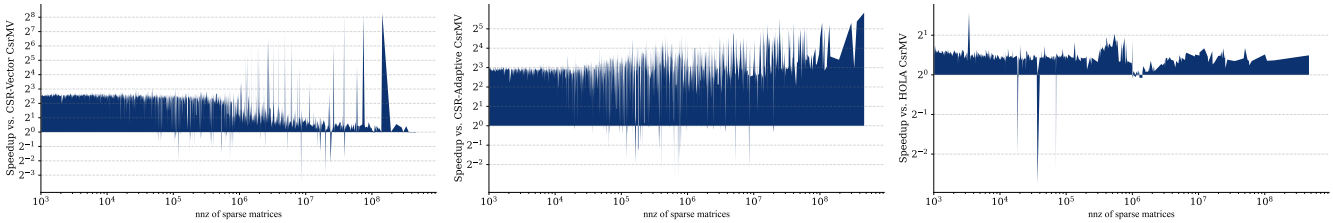
Algorithms	small matrices ($10k \leq NNZ < 300k$)					large matrices ($300k \leq NNZ < 1G$)				
	speed up of our adaptive			our adaptive	best ratio	speed up of our adaptive			our adaptive	best ratio
	max	min	mean	win		max	min	mean	win	
CSR-Vector (rocSparse)	8.80	0.27	5.23	98.43%	10/1095	319.10	0.08	5.25	96.69%	20/924
CSR-Adaptive (rocSparse)	22.76	0.26	7.10	97.64%	16/1095	56.99	0.13	10.02	96.73%	22/924
HOLA	1.77	0.15	1.33	97.37%	4/1095	2.06	0.94	1.37	95.06%	8/924
Our adaptive	-	-	-	-	1065/1095	-	-	-	-	874/924
cuSparse	1.93	0.32	1.12	81.35%	269/1095	30.05	0.12	1.46	70.81%	271/937
merge-based (cub)	19.94	0.44	2.75	98.34%	4/1095	19.53	0.09	1.72	92.43%	11/937
HOLA	5.62	0.34	2.29	97.51%	16/1095	7.55	0.11	1.49	91.79%	33/937
Our adaptive	-	-	-	-	806/1095	-	-	-	-	622/937

*our adaptive win: the percentage of matrix cases that our adaptive algorithm has better performance than the compared algorithm.
*best ratio: the percentage of matrix cases that the algorithm achieves the best performance among all algorithms.



(a) Our adaptive versus cuSparse **(b) Our adaptive versus merge-based CUB** **(c) Our adaptive versus HOLA**

Figure 6: Speedup of our adaptive method versus other algorithms on NVIDIA V100 platform

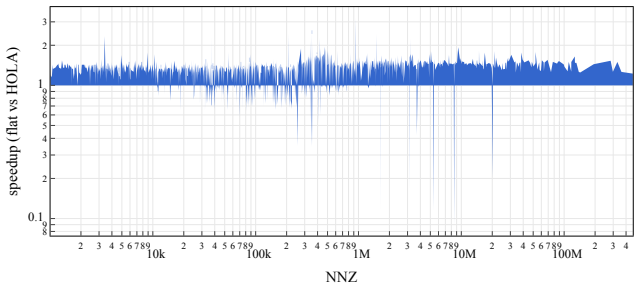


(a) Our adaptive versus RocSparse's CSR-Vector **(b) Our adaptive versus RocSparse's CSR-Adaptive** **(c) Our adaptive versus HOLA ported from the original CUDA implementation**

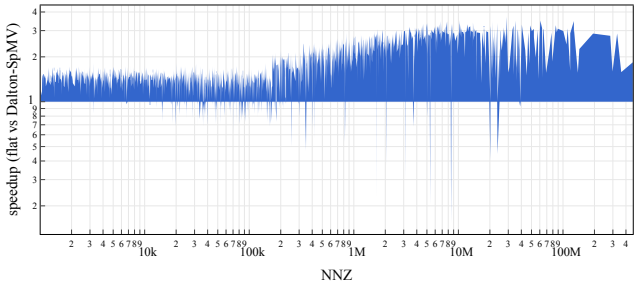
Figure 7: Speedup of our adaptive method versus other algorithms on ROCm platform

The CSR-Adaptive algorithm in rocSparse was compared with our adaptive approach. The results present an average speedup of 7.1 and 10.02 on small and large matrix cases (average speedup of 8.41 for cases of $10k \leq NNZ < 1G$), as well as a maximum speedup of 56.99, with preprocessing timing included. However, if the sequential preprocessing of CSR-Adaptive on the CPU-side is not considered, the average speedup of our method can be 0.92 and 0.97 for small and large matrix cases, respectively, which are roughly approximate for both approaches. In Figure 5(b), a subset of the matrix cases evaluated in CSR-Adaptive's paper was also tested by us (preprocessing time is not included). For the commonly tested cases, CSR-Adaptive obtains the closest performance (without preprocessing time) to our approach. Especially, for cases *boyd2*, *webbase-1M*, *ASIC_680ks* and *FullChip*, which are all largely irregular matrices, the FLOPS from CSR-Vector (rocSparse) are all close to 0, but our adaptive approach can still obtain quite high performance, in which the three cases' performance is higher than that of CSR-Adaptive.

4.1.2 SpMV Performance of Flat. The performance of the three NNZ-splitting-based methods, *flat*, HOLA and Dalton-SpMV [9], were compared in this section. Figure 8(a) demonstrates the speedup of *flat* versus HOLA on the whole dataset, with an average speedup of 1.32 and faster performance on 89.51% matrix cases. Although both *flat* and HOLA are based on NNZ-splitting, the *flat* algorithm offloads the multiplication results to LDS space and only needs to save one value from the matrix *values*, *col_index*, *x* in registers, which can largely decrease register usage. While in HOLA, the LDS space is used for storing the array *rowBits*,*flag* and *rowStarts*; thus, *R* values for each array from matrix *values*, *col_index*, *x*, and



(a) Speedup of calculation time of *flat* algorithm versus HOLA. Average speedup: 1.32; *flat* is faster than HOLA on 89.51% matrix cases.



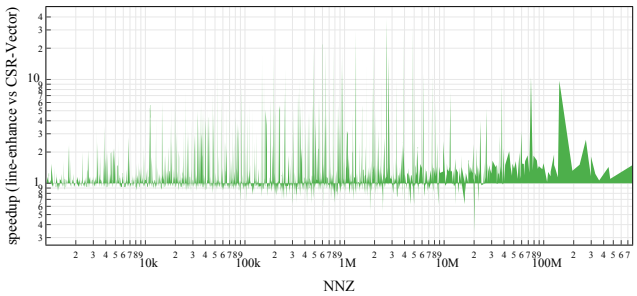
(b) Speedup of core calculation time of *flat* algorithm versus Dalton-SpMV [9] (for Dalton-SpMV, we only count the reduction phase as calculation). Average speedup: 1.71; *flat* is faster than Dalton-SpMV on 95.74% matrix cases.

Figure 8: Performance comparison of *flat* and other competitive methods under the whole dataset on NVIDIA GPU

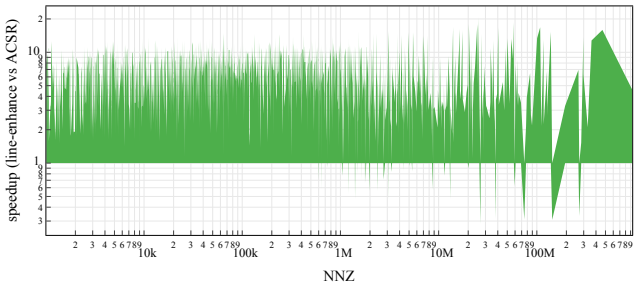
the multiplication results must be saved into registers. The register pressure of HOLA can lead to low occupancy or private memory (a memory space on device memory) spilling, and thus cause a lower performance. In our tests on the NVIDIA platform, we found that the HOLA method can spill registers to the local memory (40 registers usage plus more than 50 spill stores/loads on the NVIDIA V100 platform, while the *flat* method only uses 32 registers). Another advantage of the *flat* is that it does not require a search step to locate the starting row for each thread in a workgroup and write the records into LDS space, which can also contribute to the better performance of the *flat* algorithm.

The performance of another NNZ-splitting-based algorithm, Dalton-SpMV, was also compared with the *flat* algorithm. For the core calculation time, Figure 8(b) demonstrates the corresponding speedup for *flat* and Dalton-SpMV. Because *flat* performs reduction using the direct vector-based approach (rather than workgroup-wide and device-wide segmented scan for the reduction in Dalton-SpMV) and omits the additional overhead of row indexes generation corresponding to each non-zeros in Dalton-SpMV, *flat* can outperform Dalton-SpMV on 95.74% matrix cases and can achieve 1.71× speedup over Dalton-SpMV, although *flat* requires *AtomicAdd* operation on device memory.

4.1.3 SpMV Performance of Line-enhance. The *line-enhance* algorithm assigns the entire rows to workgroups and can perform local load balance inside the workgroup and coalesced memory accesses. To evaluate the benefit of the local load balance and coalesced



(a) Speedup of *line-enhance* algorithm versus CSR-Vector on the NVIDIA platform. Average speedup: 1.71; *line-enhance* is faster than CSR-Vector on 70.17% matrix cases.



(b) Speedup of *line-enhance* algorithm versus ACSR[4] (excluding ACSR's memory copy/allocation time and bins preparation time). Average speedup: 6.29; *line-enhance* is faster than ACSR on 98.72% matrix cases.

Figure 9: Performance comparison of *line-enhance* and other competitive methods under the whole dataset on the NVIDIA GPU

memory accesses, we compared the performance from *line-enhance* and the performance from CSR-Vector (CUDA) and the binning-based method ACSR on the NVIDIA platform. As demonstrated in Figure 9(a), compared with CSR-Vector (CUDA), the *line-enhance* method is on average 71% faster and has better performance on 70.17% matrix cases. The ACSR's performance also deteriorates, while our *line-enhance* approach increases its speedup to 6.29. *Line-enhance* is faster than ACSR in 98.72% of the whole dataset matrices.

4.2 Preprocessing Overhead

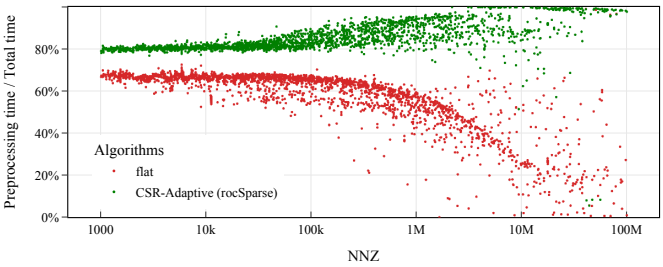


Figure 10: Preprocessing cost of our *flat* algorithm and CSR-Adaptive (rocSparse)

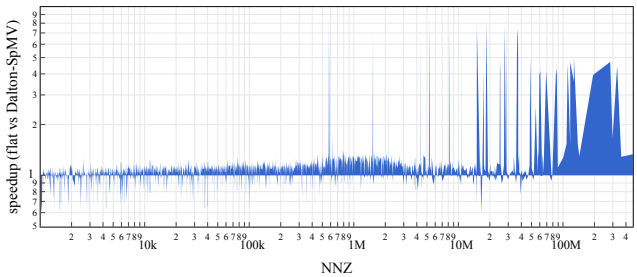


Figure 11: Speedup of preprocessing time of *flat* algorithm versus Dalton-SpMV [9]. Average speedup: 1.15; *flat* is faster than Dalton-SpMV on 86.21% matrix cases.

As the GPU-paralleled preprocessing step is involved in the *flat* algorithm, it is necessary to analyze the preprocessing overhead. Figure 10 exhibits the cost of the preprocessing time over the total time for the *flat* algorithm on the whole dataset (the red points).

In the *flat* algorithm, when $NNZ < 1\,000\,000$ ($1M$), the preprocessing time occupies more than 60% of the time of the whole SpMV on average. Conversely, the preprocessing cost can decrease to an acceptable level for large matrices (e.g., $NNZ > 1\,000\,000$), which can usually be less than 50% of the total SpMV time. In this way, for large matrices, *flat* can outperform other competing algorithms in this paper, as we described in Section 4.1. This is also a reason why the *flat* algorithm is adopted for large matrices in our adaptive algorithm selection (see Section 3.3.).

As a comparison, the preprocessing cost of CSR-Adaptive is also shown in Figure 10. It is noteworthy that the preprocessing cost for most matrix cases whose NNZ is larger than 100 000 ($100k$) can reach 80% of the total SpMV time. Moreover, the proportion of preprocessing overhead increases with the size of the matrix. In particular, for some matrices with $NNZ > 10\,000\,000$ ($10M$), the corresponding preprocessing cost can reach 97% - nearly 100% of the total SpMV time (see right top corner in Figure 10).

However, in some applications, it can amortize the preprocessing overhead by iteratively calling the SpMV routine [8, 12]. For example, if the matrix sharp (position of non-zeros) does not change, the partition results from the preprocessing step can be reused in iteration. Hence, rather than just comparing the total SpMV time (preprocessing time + SpMV kernel time), we also need to compare the kernel time directly. When compared with CSR-Adaptive in rocSparse without preprocessing time, our *flat* algorithm can still gain impressive performance on the whole dataset with a max speedup of 117.09 and 20.84% performance improvement on average. Furthermore, *flat* has better performance than CSR-Adaptive on 83.63% of matrix cases (preprocessing time is not included).

The GPU-side binary search is applied to Dalton-SpMV as the preprocessing step. Figure 11 presents the speedup of the preprocessing step (the break point generation in *flat* or partition phase in Dalton-SpMV) of *flat* versus Dalton-SpMV. *Flat*'s preprocessing method (iterate on array *row_ptr*) can achieve 1.15× speedup and can perform better on 86.21% of matrix cases when compared with Dalton-SpMV's binary search preprocessing method. This is because iterating on array *row_ptr* in the *flat*'s preprocessing method, rather than iterating on results array *bp* (Dalton-SpMV's partition phase uses this method), can achieve a better memory access pattern (e.g., coalesced memory accesses) on *row_ptr*.

4.3 Memory Bandwidth

Rather than comparing the performance of our methods with other SpMV algorithms, evaluating the bandwidth is also relevant. The reason why we do not evaluate the absolute performance (such as FLOPS) is that SpMV is a memory bandwidth-bound operation. Evaluating how close our method is to the peak memory bandwidth of a processor is more meaningful.

As discussed in previous researches[8, 25], the memory bandwidth in CSR-based SpMV can be measured by considering two extreme cases: the input vector $\boldsymbol{x}$ either is not cached or is fully cached. Equation (1) presents the formula of calculating the bandwidth bounds for the above two extreme cases, where $b_v = \text{sizeof(float64)} = 8$ and $b_i = \text{sizeof(int32)} = 4$.

$$\begin{aligned} BW_{\text{cached}} &= \frac{b_v \times (3m + NNZ) + b_i \times (m + 1 + NNZ)}{\text{kernel execution time}} \\ BW_{\text{uncached}} &= \frac{b_v \times (2m + 2NNZ) + b_i \times (m + 1 + NNZ)}{\text{kernel execution time}} \end{aligned} \tag{1}$$

Figure 12 shows the memory bandwidth our adaptive methods achieved on both the AMD and NVIDIA GPU. In each platform, the results from 60 commonly used matrices are presented. Following the assumption that the input vector $\boldsymbol{x}$ is never cached, the results indicate that 78% and 88% of cases can exceed the peak bandwidth on the ROCm and NVIDIA platforms, respectively.

When assuming that the input vector $\boldsymbol{x}$ is fully cached, a lower bound of memory bandwidth, BW_{cached}, can be given. In this case, most matrices can reach more than 70% peak memory bandwidth. In fact, the actual bandwidth can be higher than the bandwidth under this assumption due to the caches missing, especially for unordered vector $\boldsymbol{x}$ accessing.

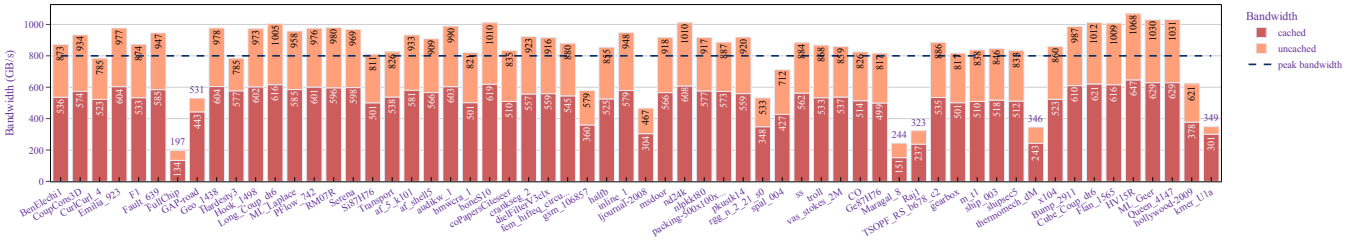
4.4 Adaptive Distribution

To evaluate the results of our adaptive strategy selection. As demonstrated in Figure 13, we present the performance distribution of *flat*, *line-enhance* and adaptive methods from the NVIDIA platform. For matrices with NNZ < 1 000 000 (1M), the adaptive strategy selection method can nearly pick all the best algorithms. For matrices with NNZ between 1 000 000 (1M) and 10 000 000 (10M), there are some selection missings (*line-enhance* has better performance, but we do not select it. See the red points on the top in Figure 13(b)). Nevertheless, for these specific matrix cases, users or developers can manually select the *line-enhance* method. Then, they can achieve more performance improvement than what the adaptive method reported in this paper. For matrices with NNZ > 10 000 000 (10M), most of the adaptive selections can achieve the highest performance.

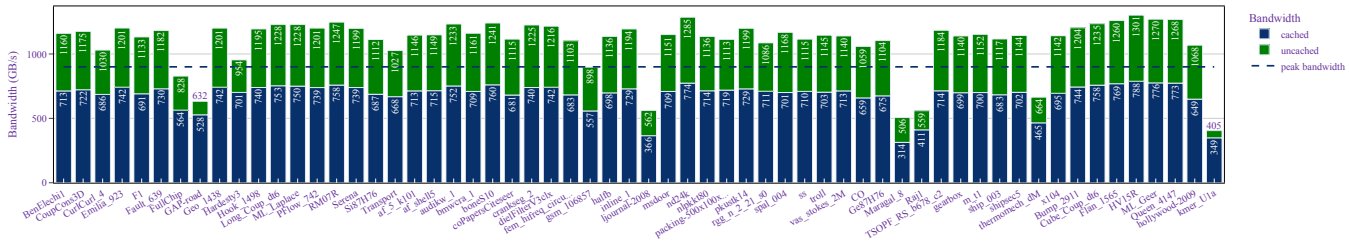
Moreover, the accuracy of our adaptive strategy selection method is also evaluated quantitatively. For the adaptive strategy selection on both the AMD and NVIDIA platforms, 79.36% and 84.98% of the strategy selections achieve the best performance, respectively.

5 CONCLUSION AND FUTURE RESEARCH

In this paper, we proposed two innovative algorithms to calculate the SpMV of the CSR format on GPUs. Both methods can efficiently utilize GPU power by designing task partitioning strategies and

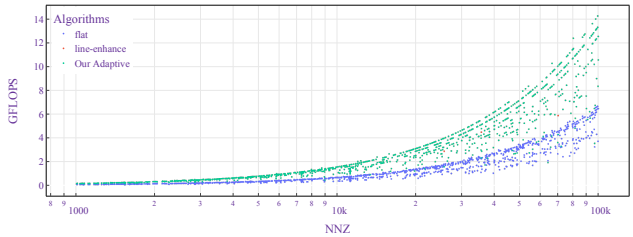


(a) Memory Bandwidth on AMD GPU (ROCm)

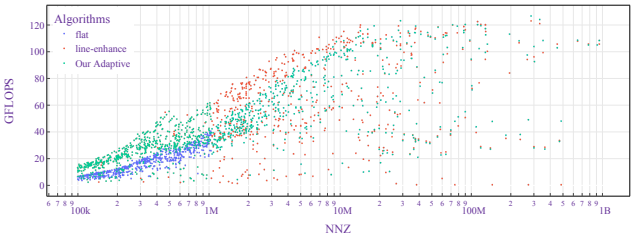


(b) Memory Bandwidth on NVIDIA Tesla V100 GPU.

Figure 12: Memory bandwidth of commonly-evaluated large-matrix datasets on the ROCm and NVIDIA platforms. The red bar on ROCm platform (or the blue bar on the NVIDIA platform) indicate the lower bound of memory bandwidth, BW_{cached}. The orange bar plus the red bar on ROCm platform (or the blue bar plus the green bar on the NVIDIA platform) indicate the upper bound of memory bandwidth, BW_{uncached}. The dashed blue lines in both subfigures are the theoretical memory bandwidth.



(a) Performance distribution on the NVIDIA GPU (part 1)



(b) Performance distribution on the NVIDIA GPU (part 2)

Figure 13: SpMV performance distribution of our adaptive method, *flat* and *line-enhance* methods on the NVIDIA platform. (The performance distribution on ROCm platform is roughly the same.)

new memory access patterns. Then, an adaptive method is proposed to combine these two SpMV algorithms together and realize quite accurate algorithm selection and parameters selection based on matrix characteristics.

Our methods were compared with the state-of-the-art algorithms and achieved quite impressive speed up with more than 20% performance improvement over previous state-of-the-art algorithms on average for large matrices. In memory bandwidth tests, the bandwidth of our approach can be very close to the peak memory-bandwidth in large matrix cases.

We found that in our preliminary experiments, our algorithms still have a performance improvement by involving some instruction optimization (e.g., loading 2 double-precision floating-point numbers at once via *global_load_dword_x*4 instruction on AMD GPU). Studying instruction optimizations can be very necessary in our future works. For the *line-enhance* method, we can also add a preprocessing step (similar to CSR-Adaptive) to assign rows to workgroups to achieve global load balance among workgroups. Optimizing CsrMV on special matrices (e.g., elements in vector $\boldsymbol{x}$ are visited in some order) can also be a significant approach to improve CsrMV performance in future works.

As a contribution to the community, the open source implementation of our algorithms and benchmarks in this paper are available at https://github.com/hpcde/spmv-acc.

ACKNOWLEDGMENTS

We thank the Sugon PRA (Priority Research Application) contest for access to the GPU machines. We also thank the HPDC'23 reviewers for their valuable suggestions and comments. The research is supported by the National Key R&D Program of China No. 2021YFB0300202 & 2021YFB0300200. Changjun Hu is the corresponding author.

REFERENCES

[1] Sarah AlAhmadi, Thaha Muhammed, Rashid Mehmood, and Aiiad Albeshri. 2020. Performance Characteristics for Sparse Matrix-Vector Multiplication on GPUs. In *Smart Infrastructure and Applications*, Rashid Mehmood, Simon See, Iyad Katib, and Imrich Chlamtac (Eds.). Springer International Publishing, Cham, 409–426. https://doi.org/10.1007/978-3-030-13705-2_17 Series Title: EAI/Springer Innovations in Communication and Computing.

[2] Atakan Altinkaynak. 2017. An efficient sparse matrix-vector multiplication on CUDA-enabled graphic processing units for finite element method simulations. *Internat. J. Numer. Methods Engrg.* 110, 1 (2017), 57–78. https://doi.org/10.1002/nme.5346

[3] Takuya Araki. 2017. Accelerating Machine Learning on Sparse Datasets with a Distributed Memory Vector Architecture. In *2017 16th International Symposium on Parallel and Distributed Computing (ISPDC)*. IEEE, Innsbruck, 112–121. https://doi.org/10.1109/ISPDC.2017.21

[4] Arash Ashari, Naser Sedaghati, John Eisenlohr, Srinivasan Parthasarath, and P. Sadayappan. 2014. Fast Sparse Matrix-Vector Multiplication on GPUs for Graph Applications. In *SC14: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, New Orleans, LA, USA, 781–792. https://doi.org/10.1109/SC.2014.69

[5] Nathan Bell and Michael Garland. 2009. Implementing sparse matrix-vector multiplication on throughput-oriented processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis - SC '09*. ACM Press, Portland, Oregon, 1. https://doi.org/10.1145/1654059.1654078

[6] Gabriele Capannini. 2011. Designing efficient parallel prefix sum algorithms for gpus. In *2011 IEEE 11th International Conference on Computer and Information Technology*. IEEE, 189–196.

[7] Yuedan Chen, Guoqing Xiao, Fan Wu, Zhuo Tang, and Keqin Li. 2020. tpSpMV: A two-phase large-scale sparse matrix-vector multiplication kernel for manycore architectures. *Information Sciences* 523 (June 2020), 279–295. https://doi.org/10.1016/j.ins.2020.03.020

[8] Mayank Daga and Joseph L. Greathouse. 2015. Structural Agnostic SpMV: Adapting CSR-Adaptive for Irregular Matrices. In *2015 IEEE 22nd International Conference on High Performance Computing (HiPC)*. IEEE, Bengaluru, India, 64–74. https://doi.org/10.1109/HiPC.2015.55

[9] Steven Dalton, Sean Baxter, Duane Merrill, Luke Olson, and Michael Garland. 2015. Optimizing Sparse Matrix Operations on GPUs Using Merge Path. In *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, Hyderabad, India, 407–416. https://doi.org/10.1109/IPDPS.2015.98

[10] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (Dec. 2011), 25 pages. https://doi.org/10.1145/2049662.2049663

[11] Istvan R eguly and Mike Giles. 2012. Efficient sparse matrix-vector multiplication on cache-based GPUs. In *2012 Innovative Parallel Computing (InPar)*. IEEE, San Jose, CA, USA, 1–12. https://doi.org/10.1109/InPar.2012.6339602

[12] Athena Elafrou, Georgios Goumas, and Nectarios Koziris. 2017. Performance Analysis and Optimization of Sparse Matrix-Vector Multiplication on Modern Multi- and Many-Core Processors. In *2017 46th International Conference on Parallel Processing (ICPP)*. 292–301. https://doi.org/10.1109/ICPP.2017.38

[13] Salvatore Filippone, Valeria Cardellini, Davide Barbieri, and Alessandro Fanfarillo. 2017. Sparse Matrix-Vector Multiplication on GPGPUs. *ACM Trans. Math. Software* 43, 4 (March 2017), 1–49. https://doi.org/10.1145/3017994

[14] Jeremy Fowers, Kalin Ovtcharov, Karin Strauss, Eric S. Chung, and Greg Stitt. 2014. A High Memory Bandwidth FPGA Accelerator for Sparse Matrix-Vector Multiplication. In *2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, Boston, MA, USA, 36–43. https://doi.org/10.1109/FCCM.2014.23

[15] Jiaquan Gao, Panpan Qi, and Guixia He. 2016. Efficient CSR-Based Sparse Matrix-Vector Multiplication on GPU. *Mathematical Problems in Engineering* 2016 (2016), 1–14. https://doi.org/10.1155/2016/4596943

[16] Michael Garland. 2008. Sparse matrix computations on manycore GPU's. In *Proceedings of the 45th annual conference on Design automation - DAC '08*. ACM Press, Anaheim, California, 2. https://doi.org/10.1145/1391469.1391473

[17] Georgios Goumas, Kornilios Kourtis, Nikos Anastopoulos, Vasileios Karakasis, and Nectarios Koziris. 2009. Performance evaluation of the sparse matrix-vector multiplication on modern architectures. *The Journal of Supercomputing* 50, 1 (Oct. 2009), 36–77. https://doi.org/10.1007/s11227-008-0251-8

[18] Joseph L. Greathouse and Mayank Daga. 2014. Efficient Sparse Matrix-Vector Multiplication on GPUs Using the CSR Storage Format. In *SC14: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, New Orleans, LA, USA, 769–780. https://doi.org/10.1109/SC.2014.68

[19] Paul Grigoraş, Pavel Burovskiy, Wayne Luk, and Spencer Sherwin. 2016. Optimising Sparse Matrix Vector multiplication for large scale FEM problems on FPGA. In *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. 1–9. https://doi.org/10.1109/FPL.2016.7577352

[20] Mark Harris, Shubhabrata Sengupta, and John D Owens. 2007. Parallel prefix sum (scan) with CUDA. *GPU gems* 3, 39 (2007), 851–876.

[21] Guixia He and Jiaquan Gao. 2016. A Novel CSR-Based Sparse Matrix-Vector Multiplication on GPUs. *Mathematical Problems in Engineering* 2016 (2016), 1–12. https://doi.org/10.1155/2016/8471283

[22] Changwan Hong, Aravind Sukumaran-Rajam, Bortik Bandyopadhyay, Jinsung Kim, Süreyya Emre Kurt, Israt Nisa, Shivani Sabhlok, Ümit V. Çatalyürek, Srinivasan Parthasarathy, and P. Sadayappan. 2018. Efficient sparse-matrix multi-vector product on GPUs. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*. ACM, Tempe Arizona, 66–79. https://doi.org/10.1145/3208040.3208062

[23] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. 2018. Dissecting the NVIDIA volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826* (2018).

[24] Mohsen Koohi Esfahani, Peter Kilpatrick, and Hans Vandierendonck. 2021. Exploiting In-Hub Temporal Locality In SpMV-Based Graph Processing. In *50th International Conference on Parallel Processing* (Lemont, IL, USA) *(ICPP 2021)*. Association for Computing Machinery, New York, NY, USA, Article 42, 10 pages. https://doi.org/10.1145/3472456.3472462

[25] Zbigniew Koza, Maciej Matyka, Sebastian Szkoda, and Łukasz Mirosław. 2014. Compressed Multirow Storage Format for Sparse Matrices on Graphics Processing Units. *SIAM Journal on Scientific Computing* 36, 2 (Jan. 2014), C219–C239. https://doi.org/10.1137/120900216

[26] Daniel Langr and Pavel Tvrdik. 2016. Evaluation Criteria for Sparse Matrix Storage Formats. *IEEE Transactions on Parallel and Distributed Systems* 27, 2 (Feb. 2016), 428–440. https://doi.org/10.1109/TPDS.2015.2401575

[27] Weifeng Liu and Brian Vinter. 2015. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing*. ACM, Newport Beach California USA, 339–350. https://doi.org/10.1145/2751205.2751209

[28] Weifeng Liu and Brian Vinter. 2015. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In *Proceedings of the 29th ACM on International Conference on Supercomputing*. ACM, Newport Beach California USA, 339–350. https://doi.org/10.1145/2751205.2751209

[29] Yongchao Liu and Bertil Schmidt. 2018. LightSpMV: Faster CUDA-Compatible Sparse Matrix-Vector Multiplication Using Compressed Sparse Rows. *Journal of Signal Processing Systems* 90, 1 (Jan. 2018), 69–86. https://doi.org/10.1007/s11265-016-1216-4

[30] Duane Merrill and Michael Garland. 2016. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In *SC16: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Salt Lake City, UT, USA, 678–689. https://doi.org/10.1109/SC.2016.57

[31] Yusuke Nagasaka, Akira Nukada, and Satoshi Matsuoka. 2016. Adaptive Multi-level Blocking Optimization for Sparse Matrix Vector Multiplication on GPU. *Procedia Computer Science* 80 (2016), 131–142. https://doi.org/10.1016/j.procs.2016.05.304

[32] Yuyao Niu, Zhengyang Lu, Meichen Dong, Zhou Jin, Weifeng Liu, and Guangming Tan. 2021. TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector Multiplication on GPUs. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, Portland, OR, USA, 68–78. https://doi.org/10.1109/IPDPS49936.2021.00016

[33] Guillermo Oyarzun, Daniel Peyrolon, Carlos Alvarez, and Xavier Martorell. 2021. An FPGA cached sparse matrix vector product (SpMV) for unstructured computational fluid dynamics simulations. arXiv:2107.12371 [physics.comp-ph]

[34] Karl Rupp, Florian Rudolf, and Josef Weinbub. 2016. ViennaCL - A High Level Linear Algebra Library for GPUs and Multi-Core CPUs. *SIAM Journal on Scientific Computing* 38 (2016), 6.

[35] Markus Steinberger, Andreas Derlery, Rhaleb Zayer, and Hans-Peter Seidel. 2016. How naive is naive SpMV on the GPU?. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, Waltham, MA, USA, 1–8. https://doi.org/10.1109/HPEC.2016.7761634

[36] Markus Steinberger, Rhaleb Zayer, and Hans-Peter Seidel. 2017. Globally homogeneous, locally adaptive sparse matrix-vector multiplication on the GPU. In *Proceedings of the International Conference on Supercomputing - ICS '17*. ACM Press, Chicago, Illinois, 1–11. https://doi.org/10.1145/3079079.3079086

[37] J. Wong, E. Kuhl, and E. Darve. 2015. A new sparse matrix vector multiplication graphics processing unit algorithm designed for finite element problems: A NEW SPARSE MATRIX VECTOR MULTIPLICATION GPU ALGORITHM. *Internat. J. Numer. Methods Engrg.* 102, 12 (June 2015), 1784–1814. https://doi.org/10.1002/nme.4865

[38] Shengen Yan, Chao Li, Yunquan Zhang, and Huiyang Zhou. 2014. yaSpMV: yet another SpMV framework on GPUs. In *Proceedings of the 19th ACM SIGPLAN symposium on Principles and practice of parallel programming - PPoPP '14*. ACM Press, Orlando, Florida, USA, 107–118. https://doi.org/10.1145/2555243.2555255

[39] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2020. Speeding Up SpMV for Power-Law Graph Analytics by Enhancing Locality & Vectorization. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Atlanta, GA, USA, 1–15. https://doi.org/10.1109/SC41405.2020.00090

[40] Haoran Zhao, Tian Xia, Chenyang Li, Wenzhe Zhao, Nanning Zheng, and Pengju Ren. 2020. Exploring Better Speculation and Data Locality in Sparse Matrix-Vector Multiplication on Intel Xeon. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE, Hartford, CT, USA, 601–609. https://doi.org/10.1109/ICCD50377.2020.00105

[41] Xavier Álvarez Farré, Andrey Gorobets, and F. Xavier Trias. 2021. A hierarchical parallel implementation for heterogeneous computing. Application to algebra-based CFD simulations on hybrid supercomputers. *Computers & Fluids* 214 (2021), 104768. https://doi.org/10.1016/j.compfluid.2020.104768